

Assignment 8.1

2303A51250

L.Indu

Batch – 18

Task Description #1: (Password Strength Validator – Apply AI in Security Context)

- Task: Apply AI to generate at least 3 assert test cases for `is_strong_password(password)` and implement the validator function.

- Requirements:

- o Password must have at least 8 characters.

- o Must include uppercase, lowercase, digit, and special character.

- o Must not contain spaces.

Example Assert Test Cases:

```
assert is_strong_password("Abcd@123") == True
```

```
assert is_strong_password("abcd123") == False
```

```
assert is_strong_password("ABCD@1234") == True
```

Expected Output #1:

- Password validation logic passing all AI-generated test cases.

Prompt :

```
#Create a Python function is_strong_password(password) that validates password strength.
```

```
#The password must:
```

```
#Have at least 8 characters
```

```
#Contain at least one uppercase letter
```

```
#Contain at least one lowercase letter
```

#Contain at least one digit
#Contain at least one special character
#Must not contain spaces
#Generate at least 3 assert test cases to verify the function.

```
password.py
password.py > ...
2  #The password must:
3  #Have at least 8 characters
4  #Contain at least one uppercase letter
5  #Contain at least one lowercase letter
6  #Contain at least one digit
7  #Contain at least one special character
8  #Must not contain spaces
9  #Generate at least 3 assert test cases to verify the function.
10 import string
11 def is_strong_password(password):
12     if len(password) < 8:
13         return False
14
15     if " " in password:
16         return False
17     has_upper = any(char.isupper() for char in password)
18     has_lower = any(char.islower() for char in password)
19     has_digit = any(char.isdigit() for char in password)
20     has_special = any(char in string.punctuation for char in password)
21     return has_upper and has_lower and has_digit and has_special
22 # Assert Test Cases
23 assert is_strong_password("Abcd@123") == True
24 assert is_strong_password("abcd123") == False
25 assert is_strong_password("ABCD@1234") == False
26 assert is_strong_password("Abc 123@") == False
27 assert is_strong_password("Strong@9") == True
28 print("All test cases passed successfully!")
```

Output

```
PS C:\Users\ANIL\Desktop\AI Assistant> & C:/Users/ANIL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/ANIL/Desktop/AI Assistant/password.py"
All test cases passed successfully!
PS C:\Users\ANIL\Desktop\AI Assistant>
```

Analysis

- 1.The password validator checks multiple security conditions.
2. The AI-generated assert test cases helped verify both valid and invalid scenarios.
3. no assertion errors occurred, the function works correctly.

Task Description #2: (Number Classification with Loops – Apply AI for Edge Case Handling)

- Task: Use AI to generate at least 3 assert test cases for a `classify_number(n)` function. Implement using loops.
- Requirements:
 - o Classify numbers as Positive, Negative, or Zero.
 - o Handle invalid inputs like strings and None.
 - o Include boundary conditions (-1, 0, 1).

Example Assert Test Cases:

```
assert classify_number(10) == "Positive"
```

```
assert classify_number(-5) == "Negative"
```

```
assert classify_number(0) == "Zero"
```

Expected Output #2:

- Classification logic passing all assert tests.

Prompt :

#Number Classification with Loops `classify_number(n)` function.
Implement using loops.

#Classify numbers as Positive, Negative, or Zero.

Handle invalid inputs like strings and None.

Include boundary conditions (-1, 0, 1).

Test cases from user input

Code:

```
numberclassification.py > ...
1  #Number Classification with Loops  classify_number(n) function. Implement using loops.
2  #Classify numbers as Positive, Negative, or Zero.
3  # Handle invalid inputs like strings and None.
4  # Include boundary conditions (-1, 0, 1).
5  def classify_number(n):
6      if n is None:
7          return "Invalid input: None"
8      if isinstance(n, str):
9          return "Invalid input: String"
10
11     if n > 0:
12         return "Positive"
13     elif n < 0:
14         return "Negative"
15     else:
16         return "Zero"
17 # Test cases from user input
18 print(classify_number(10)) # Positive
19 print(classify_number(-5)) # Negative
20 print(classify_number(0)) # Zero
21 print(classify_number(-1)) # Negative
22 print(classify_number(1)) # Positive
23 print(classify_number("Hello")) # Invalid input: String
24 print(classify_number(None)) # Invalid input: None
25 print("All test cases executed successfully!")
26
```

Output:

```
PS C:\Users\ANIL\Desktop\AI Assisant> & C:/Users/ANIL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/ANIL/Desktop/
Positive
Negative
Zero
Negative
Positive
Invalid input: String
Invalid input: None
All test cases executed successfully!
PS C:\Users\ANIL\Desktop\AI Assisant>
```

Analysis:

1. The function classifies numbers into Positive, Negative, or Zero.
2. Invalid inputs like strings and None return "Invalid Input".
3. AI-generated assert cases helped verify normal cases, edge cases, and invalid inputs.

Task Description #3: (Anagram Checker – Apply AI for String Analysis)

- Task: Use AI to generate at least 3 assert test cases for `is_anagram(str1, str2)` and implement the function.
- Requirements:
 - o Ignore case, spaces, and punctuation.
 - o Handle edge cases (empty strings, identical words).

Example Assert Test Cases:

```
assert is_anagram("listen", "silent") == True
assert is_anagram("hello", "world") == False
assert is_anagram("Dormitory", "Dirty Room") == True
```

Expected Output #3:

- Function correctly identifying anagrams and passing all AI-generated tests.

Prompt :

#Anagram Checker least 3 assert test cases for `is_anagram(str1, str2)` and implement the function.

#Ignore case, spaces, and punctuation.

Handle edge cases (empty strings, identical words).

Test cases from user input

```

anagramchecker.py > ...
1  #Anagram Checker least 3 assert test cases for is_anagram(str1, str2) and implement the function.
2  #Ignore case, spaces, and punctuation.
3  # Handle edge cases (empty strings, identical words).
4  def is_anagram(str1, str2):
5      # Remove non-alphabetic characters and convert to lowercase
6      clean_str1 = ''.join(c.lower() for c in str1 if c.isalpha())
7      clean_str2 = ''.join(c.lower() for c in str2 if c.isalpha())
8
9      # If lengths are different, they can't be anagrams
10     if len(clean_str1) != len(clean_str2):
11         return False
12
13     # If strings are identical (after cleaning), they are anagrams
14     if clean_str1 == clean_str2:
15         return True
16
17     # Sort characters and compare
18     return sorted(clean_str1) == sorted(clean_str2)
19
20 # Test cases from user input
21 assert is_anagram("Listen", "Silent") == True
22 assert is_anagram("Triangle", "Integral") == True
23 assert is_anagram("Apple", "Pabble") == False
24 assert is_anagram("Dormitory", "Dirty Room") == True
25 assert is_anagram("A gentleman", "Elegant man") == True
26 assert is_anagram("", "") == True
27 print("All test cases passed successfully!")

```

Output:

```

PS C:\Users\ANIL\Desktop\AI Assisant> & C:/Users/ANIL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/ANIL/Desktop/AI Assisant/anagramchecker.py"
All test cases passed successfully!
PS C:\Users\ANIL\Desktop\AI Assisant>

```

Analysis:

1. The function checks if two strings are anagrams.
2. AI-generated assert test cases verified both valid and invalid scenarios.
3. Since no assertion errors occurred, the function works correctly.

Task Description #4:

(Inventory Class – Apply AI to Simulate Real-World Inventory System)

- Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

- Methods:

- o add_item(name, quantity)

- o remove_item(name, quantity)

- o get_stock(name)

Example Assert Test Cases:

```
inv = Inventory()
```

```
inv.add_item("Pen", 10)
```

```
assert inv.get_stock("Pen") == 10
```

```
inv.remove_item("Pen", 5)
```

```
assert inv.get_stock("Pen") == 5
```

```
inv.add_item("Book", 3)
```

```
assert inv.get_stock("Book") == 3
```

Expected Output #4:

- Fully functional class passing all assertions.

Prompt :

#The class must include:

#add_item(name, quantity)

#remove_item(name, quantity)

#get_stock(name)

#Requirements:

#Prevent removing more stock than available.

#Handle items that do not exist.

#Generate at least 3 assert-based test cases to validate functionality.

Code :

```
inventoryclass.py > ...
1  #Create a Python class Inventory that simulates a real-world stock management system.
2  #The class must include:
3  #add_item(name, quantity)
4  #remove_item(name, quantity)
5  #get_stock(name)
6  #Requirements:
7  #Prevent removing more stock than available.
8  #Handle items that do not exist.
9  #Generate at least 3 assert-based test cases to validate functionality.
10 class Inventory:
11     def __init__(self):
12         self.stock = {}
13
14     def add_item(self, name, quantity):
15         if name in self.stock:
16             self.stock[name] += quantity
17         else:
18             self.stock[name] = quantity
19
20     def remove_item(self, name, quantity):
21         if name not in self.stock:
22             return "Item does not exist"
23         if self.stock[name] < quantity:
24             return "Not enough stock to remove"
25         self.stock[name] -= quantity
26         return "Item removed successfully"
```



```

inventoryclass.py > ...
10 class Inventory:
20     def remove_item(self, name, quantity):
26         return "Item removed successfully"
27
28     def get_stock(self, name):
29         return self.stock.get(name, "Item does not exist")
30 # Assert Test Cases
31 inventory = Inventory()
32 inventory.add_item("Apple", 50)
33 inventory.add_item("Banana", 30)
34 assert inventory.get_stock("Apple") == 50
35 assert inventory.get_stock("Banana") == 30
36 assert inventory.get_stock("Orange") == "Item does not exist"
37 assert inventory.remove_item("Apple", 20) == "Item removed successfully"
38 assert inventory.get_stock("Apple") == 30
39 assert inventory.remove_item("Apple", 40) == "Not enough stock to remove"
40 assert inventory.remove_item("Orange", 10) == "Item does not exist"
41 print("All test cases passed successfully!")
42

```

Output

```

PS C:\Users\ANIL\Desktop\AI Assisant> & C:/Users/ANIL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/ANIL/Desktop/AI Assisant/inventoryclass.py"
All test cases passed successfully!
PS C:\Users\ANIL\Desktop\AI Assisant>

```

Analysis:

1. AI-generated assert tests validated real-world scenarios like:
 - Adding stock
 - Removing stock
 - Checking non-existent items
 - Preventing over-removal
2. Since no assertion errors occurred, the system works correctly.

Task Description #5:

(Date Validation & Formatting – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for `validate_and_format_date(date_str)` to check and convert dates.
- Requirements:
 - o Validate "MM/DD/YYYY" format.
 - o Handle invalid dates.
 - o Convert valid dates to "YYYY-MM-DD".

Example Assert Test Cases:

```
assert validate_and_format_date("10/15/2023") == "2023-10-15"
```

```
assert validate_and_format_date("02/30/2023") == "Invalid Date"
```

```
assert validate_and_format_date("01/01/2024") == "2024-01-01"
```

Expected Output #5:

- Function passes all AI-generated assertions and handles edge cases.

Prompt :

#Validates the date in "MM/DD/YYYY" format

#Handles invalid dates (e.g., February 30)

#Converts valid dates into "YYYY-MM-DD" format

#Generates at least 3 assert test cases including edge cases

Assert Test Cases from user input

Code :

```
datevalidation.py > ...
2  #Validates the date in "MM/DD/YYYY" format
3  #Handles invalid dates (e.g., February 30)
4  #Converts valid dates into "YYYY-MM-DD" format
5  #Generates at least 3 assert test cases including edge cases
6  from datetime import datetime
7  def validate_and_format_date(date_str):
8      try:
9          # Validate the date format and check if it's a valid date
10         date_obj = datetime.strptime(date_str, "%m/%d/%Y")
11         # Convert to "YYYY-MM-DD" format
12         return date_obj.strftime("%Y-%m-%d")
13     except ValueError:
14         return "Invalid date"
15 # Assert Test Cases from user input
16 assert validate_and_format_date("02/28/2024") == "2024-02-28" # Valid date
17 assert validate_and_format_date("02/30/2024") == "Invalid date" # Invalid date
18 assert validate_and_format_date("13/01/2024") == "Invalid date" # Invalid month
19 assert validate_and_format_date("12/31/2024") == "2024-12-31" # Valid date
20 assert validate_and_format_date("00/10/2024") == "Invalid date" # Invalid month
21 assert validate_and_format_date("01/01/2024") == "2024-01-01" # Valid date
22 print("All test cases passed successfully!")
23
```

Output :

```
PS C:\Users\ANIL\Desktop\AI Assisant> & C:/Users/ANIL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:
All test cases passed successfully!
PS C:\Users\ANIL\Desktop\AI Assisant>
```

Analysis:

1. The function validates dates using Python's datetime module.
2. AI-generated assert cases helped verify correct and incorrect inputs.
3. Since no assertion errors occurred, the function works correctly.